claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_1d39e81c-851\agent-a54e60d6bb60df878.jsonl`
- SHA-256: `cb096897c4295ff8291c6cd35fb56741cca1f5ea17e3b77f73c624e334b63420`
- Source modified: `2026-06-18T10:29:13+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_1d39e81c-851`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`

Transcript

1. user (2026-06-18T10:20:58.686Z)

```
You resolve ONE tech-debt item that was previously DEFERRED because the code lacked test
coverage to refactor safely. You work in your OWN isolated git worktree of the badminton-
highlight-
indexer repo on THIS machine. The Python env HAS pytest/ruff/mypy/cv2/torch/numpy ? the full
offline
suite RUNS here (ignore any CLAUDE.md note saying it can't).

THE DISCIPLINE (do in this exact order):
1. SETUP:  git fetch origin  &&  git checkout -B <BRANCH> origin/master   (base MUST be
origin/master).
   Verify HEAD == origin/master and branch == <BRANCH>.
2. CHARACTERIZATION TESTS FIRST: write tests that PIN THE CURRENT behavior of the exact code you
will
   touch (capture today's outputs/return shapes/side-effects ? bugs and all; do NOT "fix"
anything).
   Run them against the UNCHANGED code and CONFIRM THEY PASS. If you cannot pin the behavior
   deterministically (e.g. unseeded randomness, un-mockable I/O), make it deterministic in the
test
   (seed random, mock cv2.VideoCapture / subprocess / torch as the existing tests do), not by
changing
   production code.
3. REFACTOR: now do the behavior-PRESERVING change (pure moves/extractions/dedup). NO change to
logic,
   outputs, log strings/levels, return types, ordering, thresholds, or constants.
4. RE-VERIFY: the SAME characterization tests must still pass unchanged.

HARD RULES:
- If you discover the item actually requires CHANGING behavior or is a redesign (not a pure
move),
  STOP ? do NOT force it. Report status='held' with the reason and what you learned.
- Edit ONLY the files named in your task (+ your new test file). Stage EXPLICIT paths (never
-A/./-u).
- Match surrounding style. Never weaken a test or lower the coverage floor to go green.

GATE (binding ? capture pass/skip counts + coverage %):
  python run_all_tests.py
  python -m ruff check .
  python -m mypy backend training
  python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70
ONLY IF ALL FOUR GREEN and coverage did NOT drop vs base (~80.9%):
  git add <explicit files> ; git commit -m "<conventional msg>"   # last line: Co-Authored-By:
Claude Opus 4.8 <noreply@anthropic.com>
  git push -u origin <BRANCH> ; gh pr create --base master --head <BRANCH> --title "..." --body
"..."   # NO merge
```

ELSE report status='gate-failed' (or 'held') with the failing output. Do NOT push a red branch.

Return the structured object: slug, branch, status ('pr-opened'|'held'|'gate-failed'|'error'),
pr_url (""+reason if none), char_tests (what you added + that they passed pre-refactor),
diffstat,
suite, ruff, mypy, coverage, behavior_note (why it's a pure move), caveat.

=== YOUR ITEM: db-execute-write-dedup (branch refactor/db-execute-write-helper) ===
ITEM: backend/infrastructure/database.py has ~14 bool-returning single-statement write methods
that
each repeat connect/commit/except-logger.error(...)/return False boilerplate with a PER-METHOD
log message.
Introduce a private `_execute_write(self, sql, params, error_msg) -> bool` and route ONLY the
bool-returning
SINGLE-STATEMENT writers through it, PRESERVING each method's exact log message text (pass it as
error_msg).
EXCLUDE: lastrowid-returning writers
(add_play_segment/add_candidate_segment/upsert_user_model/claim_due_job)
and multi-statement ones (e.g. prune_segments) ? leave them untouched.
CHAR TESTS FIRST: failure (return-False) branches are partially covered. Add tests that force a
write to
raise sqlite3.Error (e.g. monkeypatch the connection / pass a bad value) and assert the method
returns False
AND logs its specific message ? for at least 2 of the routed methods ? passing on UNCHANGED code
(the exact
per-method message must be asserted so the dedup can't silently change it). THEN dedup. If a
reviewer would
see ANY log-message change, you got it wrong.

You are running in an isolated git worktree at C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\wf_1d39e81c-851-3 (a separate working copy of the repo). Changes you
make here do NOT affect the main working directory (C:\Users\avidu\Projects\badminton-highlight-
indexer) or other agents. Work normally ? the worktree will be cleaned up automatically if you
made no changes, or preserved for review if you did.

2. assistant (2026-06-18T10:21:01.203Z)

I'll start by setting up the worktree on the correct base branch, then write characterization
tests before refactoring.

3. user (2026-06-18T10:21:05.400Z)

branch 'refactor/db-execute-write-helper' set up to track 'origin/master'.
Switched to a new branch 'refactor/db-execute-write-helper'
---HEAD---
e67add172f5b620025f31489fde2fb9ff45465ac
---ORIGIN/MASTER---
e67add172f5b620025f31489fde2fb9ff45465ac
---BRANCH---
refactor/db-execute-write-helper

4. assistant (2026-06-18T10:21:07.281Z)

HEAD matches origin/master and the branch is correct. Now let me read the target file.

5. user (2026-06-18T10:21:09.033Z)

```
1       import sqlite3
2       import json
3       import logging
4       from contextlib import contextmanager
5       from typing import List, Dict, Any, Optional, Tuple
6
```

```
7       logger = logging.getLogger(__name__)
8
9       class Database:
10          def __init__(self, db_path: str):
11              self.db_path = db_path
12              self._init_db()
13
14          @contextmanager
15          def _connect(self):
16              """Transaction scope that also CLOSES the connection.
17
18              ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
19              never closes, so every call leaked its connection to GC (lingering file
20              handles on the WAL sidecars under Windows). Internal methods use this.
21              """
22              conn = self._get_connection()
23              try:
24                  with conn:
25                      yield conn
26              finally:
27                  conn.close()
28
29          def _get_connection(self):
30              # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
31              # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
32              conn = sqlite3.connect(self.db_path, timeout=30.0)
33              conn.row_factory = sqlite3.Row
34              # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
35              conn.execute("PRAGMA foreign_keys = ON")
36              # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
37              conn.execute("PRAGMA busy_timeout = 30000")
38              try:
39                  conn.execute("PRAGMA journal_mode = WAL")
40              except sqlite3.OperationalError:
41                  pass  # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
42              return conn
43
44          @staticmethod
45          def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
46              """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runnably.
47
48              A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn:``
49              transaction, so an interrupted v3/v4 block can leave the column added while
50              ``user_version`` stays un-bumped ? and the next open would re-run the same
51              ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every
52              future ``Database(...)`` construction (``_init_db`` runs in ``__init__``).
Swallow
53              ONLY that specific error so the ladder stays crash-safe, matching the re-
runnable
54              ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
55              """
56              try:
57                  cursor.execute(ddl)
58              except sqlite3.OperationalError as exc:
59                  if "duplicate column name" not in str(exc).lower():
60                      raise
61
62          def _init_db(self):
63              """Initializes tables for videos, play_segments, and events."""
64              with self._connect() as conn:
65                  cursor = conn.cursor()
66
67                  self._create_base_tables(cursor)
68
```

```
                # Versioned migrations live here. PRAGMA user_version is the schema
                # contract ? bump it for every change and add an ordered `if version < N`
                # block below. Tests assert the expected version, so accidental drift fails
CI.
                version = cursor.execute("PRAGMA user_version").fetchone()[0]

                if version < 1:
                    self._migrate_to_v1(cursor)

                if version < 2:
                    self._migrate_to_v2(cursor)

                if version < 3:
                    self._migrate_to_v3(cursor)

                if version < 4:
                    self._migrate_to_v4(cursor)

                if version < 5:
                    self._migrate_to_v5(cursor)
                # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 6

                conn.commit()

        def _create_base_tables(self, cursor):
            """Create the videos / play_segments / events base tables (idempotent)."""
            # Videos Table
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS videos (
                    id TEXT PRIMARY KEY,
                    filepath TEXT NOT NULL,
                    processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    status TEXT NOT NULL,
                    validation_results TEXT
                )
            """)

            # Play Segments Table (Extensible metadata JSON)
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS play_segments (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    video_id TEXT NOT NULL,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    duration REAL NOT NULL,
                    server TEXT,
                    receiver TEXT,
                    metadata TEXT,
                    FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
                )
            """)

            # Events Table
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS events (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    segment_id INTEGER NOT NULL,
                    timestamp REAL NOT NULL,
                    type TEXT NOT NULL,
                    player TEXT,
                    details TEXT,
                    FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
                )
            """)
```

```

        def _migrate_to_v1(self, cursor):
            # v1: raw candidate detections (pre-confirmation), detector-agnostic.
            # Common fields are columns; detector-specific metrics go in `stats` JSON,
            # so a new segmenter reuses this table by setting its own `detector`/`stats`.
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS candidate_segments (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    video_id TEXT NOT NULL,
                    detector TEXT NOT NULL,
                    chunk_index INTEGER,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    duration REAL NOT NULL,
                    confidence REAL,
                    stats TEXT,
                    detection_params TEXT,
                    confirmed_segment_id INTEGER,
                    human_label TEXT,
                    notes TEXT,
                    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
                    FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON
DELETE SET NULL
                )
            """)
            cursor.execute("""
                CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
                    ON candidate_segments(video_id, detector)
            """)
            cursor.execute("PRAGMA user_version = 1")

        def _migrate_to_v2(self, cursor):
            # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per submitted
            # /api/process request. `status` is the EXECUTION state of the job
            # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
            # that failed validation) lives inside `result` JSON as result["status"].
            # `result` is the full sync-era response dict (incl. report paths), so the
            # observability report is the job's artifact, per the plan.
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS jobs (
                    id TEXT PRIMARY KEY,
                    video_path TEXT NOT NULL,
                    video_id TEXT,
                    segmenter TEXT,
                    player_pool TEXT,
                    max_frames INTEGER,
                    status TEXT NOT NULL DEFAULT 'queued',
                    progress TEXT,
                    error TEXT,
                    result TEXT,
                    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    started_at TIMESTAMP,
                    finished_at TIMESTAMP
                )
            """)
            cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON jobs(status)")
            cursor.execute("PRAGMA user_version = 2")

        def _migrate_to_v3(self, cursor):
            # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2). `policy` = the
            # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting' job is
due
            # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ? any worker
            # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
```

```
            # the coordinator. ALTER (not recreate) so existing v2 job rows survive; the
            # ADD COLUMNs are applied idempotently so an interrupted upgrade can re-run
them.
            self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
            self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at
TIMESTAMP")
            cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
            cursor.execute("PRAGMA user_version = 3")

        def _migrate_to_v4(self, cursor):
            # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a NULLABLE
            # `user_id` to videos + candidate_segments so a future per-user / multi-tenant
            # path can scope rows to an owner. **NULL = local install = byte-identical to
            # today** ? every existing row stays NULL and every existing query (which never
            # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows survive;
            # the ADD COLUMNs are applied idempotently so an interrupted upgrade can re-run
them.
            # Additive ONLY: no served-behavior change rides on this slice.
            self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN user_id
TEXT")
            self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD COLUMN
user_id TEXT")
            # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast path free
            # of index churn while making the eventual per-user lookups cheap.
            cursor.execute(
                "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
                "WHERE user_id IS NOT NULL")
            cursor.execute(
                "CREATE INDEX IF NOT EXISTS idx_candidates_user "
                "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
            cursor.execute("PRAGMA user_version = 4")

        def _migrate_to_v5(self, cursor):
            # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per (user_id,
            # version): the resolved per-user `fusion_config` overlay + an INLINE
            # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
evidence
            # and provenance that earned it. `is_active` pins the one row the serving bridge
            # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`). This
slice
            # only persists/loads it ? NOTHING reads it on the served path yet.
            #
            #   user_id              ? owner of the model (NULL allowed = the local install)
            #   version              ? monotonic per-user model version
            #   baseline_version     ? the shared baseline this was fit against (upgrade
switch)
            #   feature_schema_hash  ? fingerprint of the feature set; MINOR-vs-MAJOR re-fit
gate
            #   fusion_config        ? JSON per-user FusionConfig overlay (cue_flags /
thresholds)
            #   classifier_json      ? JSON LogisticRegression.to_dict() (None = config-only
model)
            #   promotion_evidence   ? JSON per_user_gate PromotionDecision (why it was
promoted)
            #   n_videos_at_fit      ? held-out-user corpus size when fit (min_n trust floor
input)
            #   is_active            ? the one resolved row for this user (partial-unique
below)
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS user_models (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    user_id TEXT,
                    version INTEGER NOT NULL DEFAULT 1,
                    baseline_version TEXT,
```

```
                    feature_schema_hash TEXT,
                    fusion_config TEXT,
                    classifier_json TEXT,
                    promotion_evidence TEXT,
                    n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
                    is_active INTEGER NOT NULL DEFAULT 0,
                    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
                )
            """)
            # One model version per user (NULLs compare distinct in SQLite, so the local
            # install can still carry multiple versioned rows; the active-row guard below
            # is what enforces a single served model).
            cursor.execute("""
                CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
                    ON user_models(user_id, version)
            """)
            # At most ONE active model per user: a partial-unique index over the active
rows.
            # (SQLite treats NULL user_id rows as distinct, so the local install's single
            # active row is still guarded ? there is one local user.)
            cursor.execute("""
                CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
                    ON user_models(user_id) WHERE is_active = 1
            """)
            cursor.execute("PRAGMA user_version = 5")

        def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
            """Register or update a video row WITHOUT touching its child segments.

            Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
            with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
            That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
            before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
            row in place; clearing segments is now an explicit, deliberate operation
            (clear_video_data / prune_segments).
            """
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "INSERT INTO videos (id, filepath, status, validation_results)
VALUES (?, ?, ?, ?) "
                        "ON CONFLICT(id) DO UPDATE SET "
                        "filepath=excluded.filepath, status=excluded.status, "
                        "validation_results=excluded.validation_results",
                        (video_id, filepath, status, json.dumps(validation_results))
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to add video: {e}")
                return False

        def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
            """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).

            Captured before a (re)segmentation so the run's new rows (id > watermark) can be
            told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
            """
            with self._connect() as conn:
                cur = conn.cursor()
```

```
                p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
                                (video_id,)).fetchone()[0]
                c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
                                (video_id,)).fetchone()[0]
                return int(p), int(c)

        def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
                           play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
                           cand_after: Optional[int] = None) -> None:
            """Delete play/candidate segments by id range (events cascade from
play_segments).

            Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
            successful run (``*_upto`` = the pre-run watermark), or drop the NEW partial
rows
            and keep the OLD ones when the run failed/produced nothing (``*_after``).
            """
            with self._connect() as conn:
                cur = conn.cursor()
                if play_upto is not None:
                    cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
                if play_after is not None:
                    cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
                if cand_upto is not None:
                    cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
                if cand_after is not None:
                    cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
                conn.commit()

        def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
            try:
                with self._connect() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
                        (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add play segment: {e}")
                return None

        def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
                        (segment_id, timestamp, event_type, player, json.dumps(details or
{}))
                    )
                    conn.commit()
```

```
                return True
            except Exception as e:
                logger.error(f"Failed to add event: {e}")
                return False

        def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
                                  chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
                                  stats: Optional[Dict[str, Any]] = None,
                                  detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
            """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
            detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
            try:
                with self._connect() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        """INSERT INTO candidate_segments
                           (video_id, detector, chunk_index, start_time, end_time, duration,
                            confidence, stats, detection_params)
                           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
                        (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
                         confidence, json.dumps(stats or {}), json.dumps(detection_params or
{}))
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add candidate segment: {e}")
                return None

        def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
            """Record that a candidate detection was confirmed as a given play_segment."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
?",
                        (segment_id, candidate_id)
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to link candidate {candidate_id} to segment
{segment_id}: {e}")
                return False

        def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
                                     only_unlabeled: bool = False) -> List[Dict[str, Any]]:
            """Fetch candidate detections for the annotation / fine-tuning workflow.
            `stats` and `detection_params` are deserialized from JSON."""
            query = "SELECT * FROM candidate_segments WHERE video_id = ?"
            params: List[Any] = [video_id]

            if detector:
                query += " AND detector = ?"
                params.append(detector)
            if only_unlabeled:
                query += " AND human_label IS NULL"

            query += " ORDER BY start_time"

```

```
            candidates = []
            with self._connect() as conn:
                rows = conn.cursor().execute(query, params).fetchall()
                for row in rows:
                    d = dict(row)
                    d["stats"] = json.loads(d["stats"] or "{}")
                    d["detection_params"] = json.loads(d["detection_params"] or "{}")
                    candidates.append(d)
            return candidates

        def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
            with self._connect() as conn:
                row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
                if row:
                    d = dict(row)
                    d["validation_results"] = json.loads(d["validation_results"] or "[]")
                    return d
            return None

        def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
            """
            Dynamically queries play segments based on filters:
            - duration_min: float
            - server: string
            - receiver: string
            - event_type: string (e.g. smash, foul)
            """
            query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
            params = [video_id]

            if filters.get("duration_min"):
                query += " AND r.duration >= ?"
                params.append(filters["duration_min"])

            if filters.get("server"):
                query += " AND r.server = ?"
                params.append(filters["server"])

            if filters.get("receiver"):
                query += " AND r.receiver = ?"
                params.append(filters["receiver"])

            if filters.get("event_type"):
                # Check if there is an event of a specific type inside this segment
                query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
                params.append(filters["event_type"])

            segments_list = []
            with self._connect() as conn:
                cursor = conn.cursor()
                rows = cursor.execute(query, params).fetchall()

                for row in rows:
                    r_dict = dict(row)
                    r_id = r_dict["id"]
                    r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")

                    # Fetch associated events
                    event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
                    r_dict["events"] = []
                    for er in event_rows:
```

```
                        e_dict = dict(er)
                        e_dict["details"] = json.loads(e_dict["details"] or "{}")
                        r_dict["events"].append(e_dict)

                    segments_list.append(r_dict)

            return segments_list

        def clear_video_data(self, video_id: str):
            with self._connect() as conn:
                conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
                conn.commit()

        # --- Async job store (DEFERRED_HARDENING_PLAN #16) -------------------------------
        # Writers follow the repo convention (swallow + logger.error, return bool); the
        # worker logs loudly on a failed transition so a job can't silently wedge.

        def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
                       player_pool: Optional[List[str]] = None,
                       max_frames: Optional[int] = None,
                       policy: Optional[Dict[str, Any]] = None) -> bool:
            """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
            (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "INSERT INTO jobs (id, video_path, segmenter, player_pool,
max_frames, status, policy) "
                        "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
                        (job_id, video_path, segmenter,
                         json.dumps(player_pool) if player_pool is not None else None,
                         max_frames, json.dumps(policy) if policy is not None else None)
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to create job {job_id}: {e}")
                return False

        def mark_job_running(self, job_id: str) -> bool:
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
                        "progress='starting' WHERE id = ?", (job_id,))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to mark job {job_id} running: {e}")
                return False

        def set_job_progress(self, job_id: str, progress: str) -> bool:
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to set progress for job {job_id}: {e}")
                return False

        def set_job_video_id(self, job_id: str, video_id: str) -> bool:
            try:
```

```
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to set video_id for job {job_id}: {e}")
                return False

        def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
            """Job EXECUTED to completion. The pipeline's own verdict (success/failed
            validation) is inside `result` ? job status 'done' means 'the worker
finished'."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET status='done', progress='finished', result=?, "
                        "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
                        (json.dumps(result), job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to mark job {job_id} done: {e}")
                return False

        def mark_job_failed(self, job_id: str, error: str) -> bool:
            """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET status='failed', error=?, "
                        "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to mark job {job_id} failed: {e}")
                return False

        def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
            """Fetch one job; `result`/`player_pool` JSON deserialized."""
            with self._connect() as conn:
                row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
(job_id,)).fetchone()
                if not row:
                    return None
                d = dict(row)
                d["result"] = json.loads(d["result"]) if d["result"] else None
                d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
None
                d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
                return d

        # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
------------------------
        def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
            """Attach/replace a job's JSON ExecutionPolicy."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET policy=? WHERE id = ?",
                        (json.dumps(policy) if policy is not None else None, job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to set policy for job {job_id}: {e}")
```

```
                return False

        def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] =
None) -> bool:
            """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
releasing the
            worker. `claim_due_job` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
            computed in SQLite's own UTC format so it compares directly to
CURRENT_TIMESTAMP."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET status='waiting', "
                        "next_poll_at=datetime('now', ?), "
                        "progress=COALESCE(?, progress) WHERE id = ?",
                        (f"+{max(0, int(delay_sec))} seconds", progress, job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to set job {job_id} waiting: {e}")
                return False

        def seconds_until_next_due(self) -> Optional[float]:
            """Seconds until the soonest parked ('waiting') job becomes due, so the worker
can
            schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
is
            already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
(0.0)."""
            try:
                with self._connect() as conn:
                    row = conn.cursor().execute(
                        "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
                        "  (julianday(next_poll_at) - julianday('now')) * 86400.0 END) AS
secs "
                        "FROM jobs WHERE status='waiting'").fetchone()
                if row is None or row["secs"] is None:
                    return None
                return max(0.0, float(row["secs"]))
            except Exception as e:
                logger.error(f"Failed to compute next-due delay: {e}")
                return None

        def claim_due_job(self) -> Optional[Dict[str, Any]]:
            """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
            flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
            double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
            first. Returns the claimed job dict, or None if nothing is runnable."""
            try:
                with self._connect() as conn:
                    cur = conn.cursor()
                    row = cur.execute(
                        "SELECT id, status FROM jobs WHERE status='queued' "
                        "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
                        "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
1").fetchone()
                    if not row:
                        return None
                    cur.execute(
                        "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
```

```
                        "WHERE id=? AND status=?", (row["id"], row["status"]))
                    conn.commit()
                    if cur.rowcount != 1:
                        return None                    # lost the race to a concurrent
worker
                return self.get_job(row["id"])
            except Exception as e:
                logger.error(f"Failed to claim a due job: {e}")
                return None

        def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
            """Crash recovery: any job still queued/running from a previous process is dead
            (the executor is in-process). Mark failed so clients see a terminal state, not a
            hang. Called once at startup. Returns the number of jobs swept."""
            try:
                with self._connect() as conn:
                    cur = conn.cursor()
                    cur.execute(
                        "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
                        "WHERE status IN ('queued', 'running')", (reason,))
                    conn.commit()
                    return cur.rowcount
            except Exception as e:
                logger.error(f"Failed to sweep stale jobs: {e}")
                return 0

        # --- Per-user model store (PERSONALIZATION_PLAN.md §2, v5) ------------------------
        # Groundwork ONLY: these persist/load the per-user model row. NOTHING on the served
        # pipeline reads them yet ? the serving bridge (backend/pipeline/personalization/
        # serving.py) resolves them, and wiring that bridge into the production decision
seam
        # (`fusion_hybrid._select_for_handoff`) is the NEXT, owner-gated PR. NULL user_id =
        # the local install. `fusion_config` / `classifier_json` / `promotion_evidence` are
        # JSON-serialised dicts; the rest are scalar columns.

        def upsert_user_model(self, user_id: Optional[str], *, version: int = 1,
                              baseline_version: Optional[str] = None,
                              feature_schema_hash: Optional[str] = None,
                              fusion_config: Optional[Dict[str, Any]] = None,
                              classifier_json: Optional[Dict[str, Any]] = None,
                              promotion_evidence: Optional[Dict[str, Any]] = None,
                              n_videos_at_fit: int = 0,
                              is_active: bool = False) -> Optional[int]:
            """Insert or replace ONE per-user model row keyed by ``(user_id, version)``.

            Idempotent on that key (UPSERT, not INSERT-OR-REPLACE: REPLACE would re-allocate
the
            rowid). When ``is_active`` is True this row becomes the user's single active
model ?
            any previously-active row for the SAME user is first cleared, so the
            ``idx_user_models_one_active`` partial-unique invariant always holds (one active
model
            per user). Returns the row id, or None on failure.
            """
            try:
                with self._connect() as conn:
                    cur = conn.cursor()
                    if is_active:
                        # Demote the current active row first (NULL user_id = local: IS NULL
match).
                        if user_id is None:
                            cur.execute(
                                "UPDATE user_models SET is_active = 0 "
                                "WHERE user_id IS NULL AND is_active = 1 AND version != ?",
```

```
                                (version,))
                        else:
                            cur.execute(
                                "UPDATE user_models SET is_active = 0 "
                                "WHERE user_id = ? AND is_active = 1 AND version != ?",
                                (user_id, version))
                    cur.execute(
                        """INSERT INTO user_models
                           (user_id, version, baseline_version, feature_schema_hash,
                            fusion_config, classifier_json, promotion_evidence,
                            n_videos_at_fit, is_active)
                           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                           ON CONFLICT(user_id, version) DO UPDATE SET
                             baseline_version=excluded.baseline_version,
                             feature_schema_hash=excluded.feature_schema_hash,
                             fusion_config=excluded.fusion_config,
                             classifier_json=excluded.classifier_json,
                             promotion_evidence=excluded.promotion_evidence,
                             n_videos_at_fit=excluded.n_videos_at_fit,
                             is_active=excluded.is_active""",
                        (user_id, version, baseline_version, feature_schema_hash,
                         json.dumps(fusion_config) if fusion_config is not None else None,
                         json.dumps(classifier_json) if classifier_json is not None else
None,
                         json.dumps(promotion_evidence) if promotion_evidence is not None
else None,
                         n_videos_at_fit, 1 if is_active else 0))
                    conn.commit()
                    rid = cur.execute(
                        "SELECT id FROM user_models WHERE "
                        + ("user_id IS NULL" if user_id is None else "user_id = ?")
                        + " AND version = ?",
                        ((version,) if user_id is None else (user_id, version))).fetchone()
                    return int(rid[0]) if rid else None
            except Exception as e:
                logger.error(f"Failed to upsert user_model (user={user_id}, v={version}):
{e}")
                return None

        def get_active_user_model(self, user_id: Optional[str]) -> Optional[Dict[str, Any]]:
            """Return the single active per-user model row for ``user_id`` (NULL = local
install),
            with ``fusion_config`` / ``classifier_json`` / ``promotion_evidence`` JSON
deserialised,
            or None if the user has no active model. Does not touch the served pipeline."""
            with self._connect() as conn:
                if user_id is None:
                    row = conn.cursor().execute(
                        "SELECT * FROM user_models WHERE user_id IS NULL AND is_active =
1").fetchone()
                else:
                    row = conn.cursor().execute(
                        "SELECT * FROM user_models WHERE user_id = ? AND is_active = 1",
                        (user_id,)).fetchone()
                if not row:
                    return None
                d = dict(row)
                for col in ("fusion_config", "classifier_json", "promotion_evidence"):
                    d[col] = json.loads(d[col]) if d[col] else None
                return d

```

6. assistant (2026-06-18T10:21:16.311Z)

Now let me identify the exact methods to route. Let me catalog the bool-returning single-

statement writers:

Routed (bool-returning, single-statement writes):
1. `add_video` ? "Failed to add video: {e}"
2. `add_event` ? "Failed to add event: {e}"
3. `link_candidate_to_segment` ? "Failed to link candidate {candidate_id} to segment {segment_id}: {e}"
4. `create_job` ? "Failed to create job {job_id}: {e}"
5. `mark_job_running` ? "Failed to mark job {job_id} running: {e}"
6. `set_job_progress` ? "Failed to set progress for job {job_id}: {e}"
7. `set_job_video_id` ? "Failed to set video_id for job {job_id}: {e}"
8. `mark_job_done` ? "Failed to mark job {job_id} done: {e}"
9. `mark_job_failed` ? "Failed to mark job {job_id} failed: {e}"
10. `set_job_policy` ? "Failed to set policy for job {job_id}: {e}"
11. `set_job_waiting` ? "Failed to set job {job_id} waiting: {e}"

Excluded (lastrowid): add_play_segment, add_candidate_segment, upsert_user_model, claim_due_job.
Excluded (multi-statement/non-bool): prune_segments (None), fail_stale_jobs (int), seconds_until_next_due (reads).

Note `seconds_until_next_due` returns Optional[float] (a read). Let me check existing tests for this module to mirror style.

7. user (2026-06-18T10:21:16.846Z)

tests\test_database.py

8. user (2026-06-18T10:21:17.077Z)

scratch\test_ai_segmenter.py
scratch\test_thresholds.py
tests\test_ablation.py
tests\test_annotation_providers.py
tests\test_api_endpoints.py
tests\test_async_jobs.py
tests\test_audio_features.py
tests\test_automated_provider.py
tests\test_base.py
tests\test_calibrate_local.py
tests\test_calibrate_wasb.py
tests\test_calibration.py
tests\test_cleanup_caches.py
tests\test_compiler.py
tests\test_config.py
tests\test_config_wasb.py
tests\test_database.py
tests\test_detector_keying.py
tests\test_distill_local.py
tests\test_eval_annotations.py
tests\test_eval_batch.py
tests\test_eval_classifier.py
tests\test_eval_harness.py
tests\test_eval_metrics.py
tests\test_eval_partial_labels.py
tests\test_eval_regression.py
tests\test_eval_stats.py
tests\test_eval_training.py
tests\test_execution_policy.py
tests\test_experiment.py
tests\test_experiment_honest.py
tests\test_export_wasb_segments.py
tests\test_fusion_compare.py
tests\test_fusion_features.py
tests\test_fusion_golden_telemetry.py

```
tests\test_fusion_hybrid.py
tests\test_gemini.py
tests\test_gemini_refine.py
tests\test_generate_highlights.py
tests\test_geometry.py
tests\test_golden_fixtures.py
tests\test_golden_real_fixtures.py
tests\test_gt_loader.py
tests\test_hashing.py
tests\test_hit_detector.py
tests\test_import_smoke.py
tests\test_job_self_scheduling.py
tests\test_job_worker_loop.py
tests\test_logging_setup.py
tests\test_nightly_regression.py
tests\test_path_safety.py
tests\test_per_user_eval.py
tests\test_per_user_gate.py
tests\test_per_user_regression.py
tests\test_personalization_serving.py
tests\test_person_detector.py
tests\test_promotion.py
tests\test_provider_gemini.py
tests\test_providers.py
tests\test_rally_gate.py
tests\test_rally_seg_eval.py
tests\test_rally_seq_proto.py
tests\test_rally_slicer.py
tests\test_rally_summary.py
tests\test_reingest_safety.py
tests\test_reliability_hardening.py
tests\test_reporting.py
tests\test_reporting_harvest.py
tests\test_reporting_spec_faq.py
tests\test_results.py
tests\test_run_signals.py
tests\test_run_telemetry.py
tests\test_score_calibration.py
tests\test_segmentation_metrics.py
tests\test_serve_contrast.py
tests\test_served_gate_a_regression.py
tests\test_splits.py
tests\test_sport_profiles.py
tests\test_tolerance_metrics.py
tests\test_tracknet_hybrid.py
tests\test_tracknet_runner.py
tests\test_training_gen0.py
tests\test_trajectory_hybrid_equivalence.py
tests\test_upload_download.py
tests\test_validators_base.py
tests\test_variant_tuning.py
tests\test_video.py
tests\test_wasb_cache.py
tests\test_wasb_runner.py
tests\test_watermark.py
tests\test_windowing.py
tests\test_wsl_tilde_quoting.py
tests\test_yolo_hybrid.py
```

9. user (2026-06-18T10:21:20.107Z)

```
1       import sqlite3
2       import pytest
3       from backend.infrastructure.database import Database
4
```

```
    def test_database_init(tmp_path):
        db_path = tmp_path / "test.db"
        Database(str(db_path))  # instantiation creates the schema

        conn = sqlite3.connect(str(db_path))
        cursor = conn.cursor()
        cursor.execute("SELECT name FROM sqlite_master WHERE type='table'")
        tables = [r[0] for r in cursor.fetchall()]
        assert "videos" in tables
        assert "play_segments" in tables
        conn.close()

    def test_database_video(tmp_path):
        db = Database(str(tmp_path / "test.db"))

        vid_added = db.add_video("vid1", "vid1.mp4", "pending", [{"val": 1}])
        assert vid_added is True

        video = db.get_video("vid1")
        assert video["id"] == "vid1"
        assert video["filepath"] == "vid1.mp4"
        assert video["status"] == "pending"
        assert video["validation_results"] == [{"val": 1}]

        db.clear_video_data("vid1")
        assert db.get_video("vid1") is None

    def test_database_play_segment(tmp_path):
        db = Database(str(tmp_path / "test.db"))
        db.add_video("vid1", "vid1.mp4", "pending", [])

        sid = db.add_play_segment("vid1", 10.0, 20.0, "PlayerA", "PlayerB", {"foo": "bar"})
        assert sid is not None

        db.add_event(sid, 15.0, "smash", "PlayerA", {"speed": 300})

        segments = db.query_play_segments("vid1", {"server": "PlayerA"})
        assert len(segments) == 1
        assert segments[0]["id"] == sid
        assert segments[0]["start_time"] == 10.0
        assert segments[0]["server"] == "PlayerA"
        assert segments[0]["metadata"]["foo"] == "bar"

        assert len(segments[0]["events"]) == 1
        assert segments[0]["events"][0]["type"] == "smash"
        assert segments[0]["events"][0]["details"]["speed"] == 300


    # --- candidate_segments schema + behavior ----------------------------------

    # The schema contract: any future change to candidate_segments columns must
    # consciously update this set, otherwise the test below fails (regression guard).
    EXPECTED_CANDIDATE_COLUMNS = {
        "id", "video_id", "detector", "chunk_index", "start_time", "end_time",
        "duration", "confidence", "stats", "detection_params",
        "confirmed_segment_id", "human_label", "notes", "created_at",
        "user_id",  # v4: personalization groundwork (nullable; NULL = local install)
    }


    def test_candidate_schema_contract(tmp_path):
        Database(str(tmp_path / "test.db"))  # instantiation creates the schema
        conn = sqlite3.connect(str(tmp_path / "test.db"))
        cols = conn.execute("PRAGMA table_info(candidate_segments)").fetchall()
        conn.close()
```

```
70
71          names = {c[1] for c in cols}
72          assert names == EXPECTED_CANDIDATE_COLUMNS
73
74          # NOT NULL / PK flags on the load-bearing columns (cid, name, type, notnull, dflt,
pk)
75          by_name = {c[1]: c for c in cols}
76          assert by_name["id"][5] == 1  # primary key
77          for col in ("video_id", "detector", "start_time", "end_time", "duration"):
78              assert by_name[col][3] == 1, f"{col} should be NOT NULL"
79
80
81      def test_schema_version_is_current(tmp_path):
82          db_path = tmp_path / "test.db"
83          Database(str(db_path))
84          conn = sqlite3.connect(str(db_path))
85          version = conn.execute("PRAGMA user_version").fetchone()[0]
86          conn.close()
87          # v5 = per-user model store (PERSONALIZATION_PLAN.md §2). Bump with each migration.
88          assert version == 5
89
90
91      def test_init_db_idempotent_and_upgrades_old_db(tmp_path):
92          db_path = tmp_path / "legacy.db"
93          # Simulate a pre-v1 database that only has the old tables and no version.
94          conn = sqlite3.connect(str(db_path))
95          conn.execute("CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT, status
TEXT)")
96          conn.commit()
97          conn.close()
98
99          # Opening with the new Database should add candidate_segments without error...
100         Database(str(db_path))
101         # ...and running init again must be a no-op (idempotent).
102         Database(str(db_path))
103
104         conn = sqlite3.connect(str(db_path))
105         tables = [r[0] for r in conn.execute(
106             "SELECT name FROM sqlite_master WHERE type='table'").fetchall()]
107         version = conn.execute("PRAGMA user_version").fetchone()[0]
108         conn.close()
109         assert "candidate_segments" in tables
110         assert "jobs" in tables  # v2 (async job model, #16)
111         assert "user_models" in tables  # v5 (per-user model store)
112         assert version == 5      # v5 = per-user model store (PERSONALIZATION_PLAN.md §2)
113
114
115     def test_candidate_round_trip(tmp_path):
116         db = Database(str(tmp_path / "test.db"))
117         db.add_video("vid1", "vid1.mp4", "processed", [])
118
119         cid = db.add_candidate_segment(
120             "vid1", "yolo_hybrid", 10.0, 25.0, chunk_index=0, confidence=0.42,
121             stats={"peak_velocity": 0.8, "track_id_count": 2},
122             detection_params={"frame_skip": 3},
123         )
124         assert cid is not None
125
126         cands = db.query_candidate_segments("vid1")
127         assert len(cands) == 1
128         c = cands[0]
129         assert c["detector"] == "yolo_hybrid"
130         assert c["start_time"] == 10.0
131         assert c["duration"] == 15.0
132         assert c["confidence"] == 0.42
```

```
        assert c["stats"]["peak_velocity"] == 0.8
        assert c["detection_params"]["frame_skip"] == 3
        assert c["confirmed_segment_id"] is None

        # detector filter
        assert db.query_candidate_segments("vid1", detector="motion") == []
        assert len(db.query_candidate_segments("vid1", detector="yolo_hybrid")) == 1


    def test_candidate_link_and_unlabeled_filter(tmp_path):
        db = Database(str(tmp_path / "test.db"))
        db.add_video("vid1", "vid1.mp4", "processed", [])

        cid = db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
        sid = db.add_play_segment("vid1", 1.0, 4.0)

        assert db.link_candidate_to_segment(cid, sid) is True
        assert db.query_candidate_segments("vid1")[0]["confirmed_segment_id"] == sid

        # only_unlabeled returns it while human_label is NULL
        assert len(db.query_candidate_segments("vid1", only_unlabeled=True)) == 1
        with db._get_connection() as conn:
            conn.execute("UPDATE candidate_segments SET human_label = ? WHERE id = ?",
                         ("true_rally", cid))
            conn.commit()
        assert db.query_candidate_segments("vid1", only_unlabeled=True) == []


    def test_candidate_cascade_on_video_delete(tmp_path):
        db = Database(str(tmp_path / "test.db"))
        db.add_video("vid1", "vid1.mp4", "processed", [])
        db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)

        db.clear_video_data("vid1")
        assert db.query_candidate_segments("vid1") == []


    # --- v4: personalization groundwork (nullable user_id; NULL = local = unchanged) -----

    def _table_cols(db_path, table):
        conn = sqlite3.connect(str(db_path))
        cols = {c[1] for c in conn.execute(f"PRAGMA table_info({table})").fetchall()}
        conn.close()
        return cols


    def test_v4_user_id_columns_present_and_nullable(tmp_path):
        db_path = tmp_path / "test.db"
        Database(str(db_path))
        assert "user_id" in _table_cols(db_path, "videos")
        assert "user_id" in _table_cols(db_path, "candidate_segments")

        # The column is NULLABLE: PRAGMA table_info notnull flag must be 0.
        conn = sqlite3.connect(str(db_path))
        for table in ("videos", "candidate_segments"):
            by_name = {c[1]: c for c in conn.execute(f"PRAGMA
table_info({table})").fetchall()}
            assert by_name["user_id"][3] == 0, f"{table}.user_id must be nullable"
        conn.close()


    def test_v4_existing_writers_leave_user_id_null(tmp_path):
        """NULL user_id back-compat: add_video / add_candidate_segment never set user_id, so
every
        row a current writer produces is local (NULL) ? byte-identical to pre-v4
```

```
behaviour."""
196         db_path = tmp_path / "test.db"
197         db = Database(str(db_path))
198         db.add_video("vid1", "vid1.mp4", "processed", [])
199         db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
200
201         conn = sqlite3.connect(str(db_path))
202         vu = conn.execute("SELECT user_id FROM videos WHERE id = 'vid1'").fetchone()[0]
203         cu = conn.execute("SELECT user_id FROM candidate_segments WHERE video_id =
'vid1'").fetchone()[0]
204         conn.close()
205         assert vu is None
206         assert cu is None
207         # Existing readers are unaffected (no user_id filter in the query path).
208         assert len(db.query_candidate_segments("vid1")) == 1
209
210
211     def test_migration_v3_db_upgrades_to_v5_preserving_rows(tmp_path):
212         """A real v3 DB (jobs + candidates already populated) upgrades cleanly to v5: the
new
213         columns/tables appear, user_id back-fills to NULL, and prior rows survive the
ALTERs."""
214         db_path = tmp_path / "legacy_v3.db"
215         db = Database(str(db_path))  # fresh DB is born at the current version; simulate a
v3 state
216         # Seed pre-upgrade data, then force the version back to 3 and re-open to run v4+v5.
217         db.add_video("vidA", "a.mp4", "processed", [{"v": 1}])
218         db.add_candidate_segment("vidA", "wasb_hybrid", 2.0, 9.0, stats={"net_crossings":
3})
219         conn = sqlite3.connect(str(db_path))
220         conn.execute("DROP INDEX IF EXISTS idx_videos_user")
221         conn.execute("DROP INDEX IF EXISTS idx_candidates_user")
222         conn.execute("DROP TABLE IF EXISTS user_models")
223         # Recreate v3-shaped tables WITHOUT user_id by rebuilding (simplest: just rewind
version
224         # and drop the v4/v5 artifacts that a true v3 DB wouldn't have).
225         conn.execute("ALTER TABLE videos DROP COLUMN user_id")
226         conn.execute("ALTER TABLE candidate_segments DROP COLUMN user_id")
227         conn.execute("PRAGMA user_version = 3")
228         conn.commit()
229         conn.close()
230         assert "user_id" not in _table_cols(db_path, "videos")
231
232         # Re-open: the migration ladder must carry the v3 DB up to v5 without data loss.
233         db2 = Database(str(db_path))
234         conn = sqlite3.connect(str(db_path))
235         assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
236         assert "user_models" in {r[0] for r in conn.execute(
237             "SELECT name FROM sqlite_master WHERE type='table'").fetchall()}
238         conn.close()
239         assert "user_id" in _table_cols(db_path, "videos")
240         assert "user_id" in _table_cols(db_path, "candidate_segments")
241
242         v = db2.get_video("vidA")
243         assert v is not None and v["validation_results"] == [{"v": 1}]
244         cands = db2.query_candidate_segments("vidA")
245         assert len(cands) == 1 and cands[0]["stats"]["net_crossings"] == 3
246
247
248     def test_migration_idempotent_on_interrupted_upgrade(tmp_path):
249         """An INTERRUPTED v3/v4 upgrade must not brick the DB. A DDL ``ALTER`` auto-commits
250         on its own, so a crash after the ADD COLUMNs but before the ``PRAGMA user_version``
251         bump leaves the columns present while the version is stale ? re-opening re-runs the
252         bare ALTERs. The ladder must tolerate that (re-runnable, like the v1/v2/v5 blocks)
253         instead of raising ``OperationalError: duplicate column name``."""
```

```
        db_path = tmp_path / "interrupted.db"
        Database(str(db_path))  # fully migrate to the current version (v3/v4 columns all
present)

        # Simulate the interruption: the v3/v4 ADD COLUMNs already landed and auto-
committed,
        # but the process died before the version bump, so user_version is stuck at 2.
        conn = sqlite3.connect(str(db_path))
        conn.execute("DROP TABLE IF EXISTS user_models")          # a true v2 DB lacks the
v5 table
        conn.execute("PRAGMA user_version = 2")
        conn.commit()
        conn.close()

        # Re-opening re-runs v3 (jobs.policy/next_poll_at) and v4 (videos/candidate_segments
        # .user_id) over columns that ALREADY exist ? must NOT raise, and must reach v5.
        Database(str(db_path))
        conn = sqlite3.connect(str(db_path))
        assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
        conn.close()
        # Columns/tables are all present after the clean re-apply.
        assert "policy" in _table_cols(db_path, "jobs")
        assert "user_id" in _table_cols(db_path, "videos")
        assert "user_id" in _table_cols(db_path, "candidate_segments")
        conn = sqlite3.connect(str(db_path))
        assert "user_models" in {r[0] for r in conn.execute(
            "SELECT name FROM sqlite_master WHERE type='table'").fetchall()}
        conn.close()


    def test_add_column_idempotent_swallows_only_duplicate(tmp_path):
        """The idempotent ADD COLUMN helper swallows ONLY 'duplicate column name'. Any other
        OperationalError (e.g. a genuinely broken migration against a missing table) must
still
        propagate ? the swallow is narrow, not a blanket except."""
        db_path = tmp_path / "helper.db"
        Database(str(db_path))                      # current schema; jobs.policy already
exists
        conn = sqlite3.connect(str(db_path))
        cur = conn.cursor()
        # Re-adding an existing column is tolerated (the crash-safety path).
        Database._add_column_idempotent(cur, "ALTER TABLE jobs ADD COLUMN policy TEXT")
        # An unrelated OperationalError (no such table) is NOT masked.
        with pytest.raises(sqlite3.OperationalError):
            Database._add_column_idempotent(cur, "ALTER TABLE nope ADD COLUMN x TEXT")
        conn.close()


    # --- v5: per-user model store CRUD ------------------------------------------------

    def test_user_models_schema_contract(tmp_path):
        db_path = tmp_path / "test.db"
        Database(str(db_path))
        cols = _table_cols(db_path, "user_models")
        assert cols == {
            "id", "user_id", "version", "baseline_version", "feature_schema_hash",
            "fusion_config", "classifier_json", "promotion_evidence",
            "n_videos_at_fit", "is_active", "created_at",
        }


    def test_upsert_and_get_active_user_model_round_trip(tmp_path):
        db = Database(str(tmp_path / "test.db"))
        rid = db.upsert_user_model(
            "user-1", version=1, baseline_version="base-2026-06",
```

```
            feature_schema_hash="abc123",
            fusion_config={"min_crossings": 3, "cue_flags": {"person": True}},
            classifier_json={"model": "logreg", "w": [0.1, 0.2], "b": 0.0},
            promotion_evidence={"promote": True, "n": 6},
            n_videos_at_fit=6, is_active=True,
        )
        assert rid is not None

        m = db.get_active_user_model("user-1")
        assert m is not None
        assert m["user_id"] == "user-1"
        assert m["version"] == 1
        assert m["baseline_version"] == "base-2026-06"
        assert m["feature_schema_hash"] == "abc123"
        assert m["fusion_config"] == {"min_crossings": 3, "cue_flags": {"person": True}}
        assert m["classifier_json"]["w"] == [0.1, 0.2]
        assert m["promotion_evidence"]["promote"] is True
        assert m["n_videos_at_fit"] == 6
        assert m["is_active"] == 1


    def test_get_active_user_model_none_when_absent_or_inactive(tmp_path):
        db = Database(str(tmp_path / "test.db"))
        assert db.get_active_user_model("nobody") is None

        # An inactive model is not returned as active.
        db.upsert_user_model("user-2", version=1, is_active=False)
        assert db.get_active_user_model("user-2") is None


    def test_upsert_is_idempotent_on_user_version(tmp_path):
        db = Database(str(tmp_path / "test.db"))
        rid1 = db.upsert_user_model("user-3", version=1, n_videos_at_fit=4, is_active=True)
        rid2 = db.upsert_user_model(
            "user-3", version=1, n_videos_at_fit=7, is_active=True,
            fusion_config={"min_crossings": 5},
        )
        # Same (user, version) ? same row updated in place, not a duplicate.
        assert rid1 == rid2
        m = db.get_active_user_model("user-3")
        assert m["n_videos_at_fit"] == 7
        assert m["fusion_config"] == {"min_crossings": 5}

        conn = sqlite3.connect(str(tmp_path / "test.db"))
        count = conn.execute(
            "SELECT COUNT(*) FROM user_models WHERE user_id = 'user-3'").fetchone()[0]
        conn.close()
        assert count == 1


    def test_activating_new_version_demotes_prior_active(tmp_path):
        db = Database(str(tmp_path / "test.db"))
        db.upsert_user_model("user-4", version=1, n_videos_at_fit=5, is_active=True)
        db.upsert_user_model("user-4", version=2, n_videos_at_fit=9, is_active=True)

        active = db.get_active_user_model("user-4")
        assert active["version"] == 2  # the newer model is the single active one

        # Exactly one active row for the user (the partial-unique invariant holds).
        conn = sqlite3.connect(str(tmp_path / "test.db"))
        n_active = conn.execute(
            "SELECT COUNT(*) FROM user_models WHERE user_id = 'user-4' AND is_active =
1").fetchone()[0]
        conn.close()
        assert n_active == 1
```

```
378
379
380     def test_null_user_id_local_install_round_trip(tmp_path):
381         """NULL user_id = the local install. upsert/get must treat IS NULL correctly (not
'=NULL')."""
382         db = Database(str(tmp_path / "test.db"))
383         db.upsert_user_model(None, version=1, fusion_config={"min_crossings": 2},
384                              n_videos_at_fit=5, is_active=True)
385         m = db.get_active_user_model(None)
386         assert m is not None
387         assert m["user_id"] is None
388         assert m["fusion_config"] == {"min_crossings": 2}
389         # A named user does NOT see the local row.
390         assert db.get_active_user_model("user-x") is None
391
392
393     def test_user_model_json_columns_default_none(tmp_path):
394         """A config-only model (no classifier) stores NULL classifier_json and deserialises
to None."""
395         db = Database(str(tmp_path / "test.db"))
396         db.upsert_user_model("user-5", version=1, fusion_config={"min_crossings": 4},
397                              n_videos_at_fit=5, is_active=True)
398         m = db.get_active_user_model("user-5")
399         assert m["classifier_json"] is None
400         assert m["promotion_evidence"] is None
401         assert m["fusion_config"] == {"min_crossings": 4}
402
```

10. assistant (2026-06-18T10:21:23.082Z)

Now I understand the existing test style. Let me check existing failure-branch coverage in the async jobs test to understand how errors are forced.

11. user (2026-06-18T10:21:25.486Z)

```
35:def env(tmp_path, monkeypatch):
38:    monkeypatch.setattr(m, "db_instance", db)
39:    monkeypatch.setattr(m, "global_config", cfg)
40:    monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
42:    return client, db, tmp_path, monkeypatch
81:    with db._get_connection() as conn:
106:    with db._get_connection() as c:
120:def test_parity_job_model_with_direct_processing(env, tmp_path, monkeypatch):
219:    with db._get_connection() as c:
```

12. assistant (2026-06-18T10:21:27.813Z)

Let me check how existing tests force write failures to find the cleanest mock pattern.

13. user (2026-06-18T10:21:29.574Z)

```
32:    conn = db._get_connection()
```

14. assistant (2026-06-18T10:21:33.646Z)

Let me check whether any existing test exercises the return-False branch of these writers, and how caplog is used in the repo.

15. user (2026-06-18T10:21:34.937Z)

```
Found 12 files
tests\test_watermark.py
tests\test_tracknet_runner.py
tests\test_provider_gemini.py
```

```
tests\test_gt_loader.py
tests\test_gemini.py
tests\test_fusion_golden_telemetry.py
tests\test_experiment.py
tests\test_execution_policy.py
tests\test_database.py
tests\test_config.py
tests\test_automated_provider.py
tests\test_base.py
```

16. assistant (2026-06-18T10:21:36.857Z)

Wait, test_database.py already uses caplog? Let me check.

17. user (2026-06-18T10:21:38.116Z)

No matches found

18. assistant (2026-06-18T10:21:40.675Z)

The grep matched substrings in other terms. Let me look at how caplog + the database logger is used in test_execution_policy.py as a model for asserting log messages.

19. user (2026-06-18T10:21:42.171Z)

```
5:are harmless). Logging discipline (polls at DEBUG, transitions at INFO) is asserted via
caplog.
7:import logging
21:LOGGER = logging.getLogger("test.execution.policy")
61:    assert run_bounded(lambda: 42, DEFAULT_POLICY, label="ok", sleep=clock.sleep,
logger=LOGGER) == 42
76:    assert run_bounded(flaky, pol, label="flaky", sleep=clock.sleep, logger=LOGGER) == "ok"
89:        run_bounded(always_fail, pol, label="bad", sleep=clock.sleep, logger=LOGGER)
97:        run_bounded(lambda: time.sleep(2.0), pol, label="hang", sleep=clock.sleep,
logger=LOGGER)
111:    assert run_bounded(hang_once, pol, label="h", sleep=clock.sleep, logger=LOGGER) ==
"recovered"
125:        run_bounded(fail, pol, label="abort", sleep=clock.sleep, logger=LOGGER)
134:                       sleep=clock.sleep, monotonic=clock.monotonic, logger=LOGGER) == "done"
148:                       logger=LOGGER) == "value"
157:                   monotonic=clock.monotonic, logger=LOGGER)
161:def test_poll_logging_is_debug_not_info(caplog):
170:    with caplog.at_level(logging.DEBUG, logger="test.execution.policy"):
172:                   monotonic=clock.monotonic, logger=LOGGER)
173:    debug = [r for r in caplog.records if r.levelno == logging.DEBUG and "poll" in
r.message]
174:    info = [r for r in caplog.records if r.levelno == logging.INFO]
```

20. assistant (2026-06-18T10:21:49.541Z)

Now I understand the caplog pattern. The database module's logger is `backend.infrastructure.database`.

Now let me write characterization tests. I'll force a write to raise `sqlite3.Error` by monkeypatching `_connect` to yield a connection whose cursor's `execute` raises ? but the cleanest approach that pins the *exact* per-method log message is to pass a value that genuinely fails at the SQL layer.

The most robust deterministic approach: monkeypatch `_get_connection` so that the cursor raises `sqlite3.Error` on execute. Let me design it to capture the message exactly via caplog. I'll write tests for at least 2 routed methods (`mark_job_running` and `create_job`, plus `add_video`, `add_event` for breadth). Let me verify the logger name and that `propagate` is default True.

21. user (2026-06-18T10:21:51.073Z)

```
backend.infrastructure.database
propagate True
```

22. assistant (2026-06-18T10:21:55.351Z)

Now I'll write the characterization tests. I'll force a `sqlite3.Error` by monkeypatching `db._get_connection` to return a connection whose cursor raises on `execute`. This deterministically drives the except branch for the routed bool-returning writers, and I'll assert both the return value (`False`) and the exact per-method log message via caplog.

I'll cover 4 routed methods to exceed the "at least 2" requirement, including ones with format args (`create_job` with `{job_id}`, `link_candidate_to_segment` with two args), since those are the ones most likely to silently change if the error_msg routing is mishandled.

23. user (2026-06-18T10:21:56.371Z)

```
1       import sqlite3
2       import pytest
3       from backend.infrastructure.database import Database
4
```

24. user (2026-06-18T10:21:59.585Z)

```
The file C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/wf_1d39e81c-851-3/tests/test_database.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

25. assistant (2026-06-18T10:22:01.736Z)

Now append the characterization tests at the end of the file.

26. user (2026-06-18T10:22:18.278Z)

```
The file C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/wf_1d39e81c-851-3/tests/test_database.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

27. assistant (2026-06-18T10:22:20.607Z)

Now run these characterization tests against the UNCHANGED production code to confirm they pass.

28. user (2026-06-18T10:22:24.048Z)

```
......                                                                   [100%]
6 passed, 21 deselected in 0.28s
```

29. assistant (2026-06-18T10:22:26.604Z)

All 6 characterization tests pass against unchanged code. Let me also run the full test_database.py file to confirm nothing else broke.

30. user (2026-06-18T10:22:29.495Z)

```
.........................                                                [100%]
27 passed in 1.01s
```

31. assistant (2026-06-18T10:22:38.964Z)

All green pre-refactor (21 existing + 6 new). Now the behavior-preserving refactor. I'll introduce `_execute_write` and route the 11 bool-returning single-statement writers through it, preserving each method's exact log message text by passing it as `error_msg`.

Let me add the helper after `_get_connection` (and before `_add_column_idempotent` to keep connection-related helpers grouped). I'll place it right after `_get_connection`.

32. user (2026-06-18T10:22:44.641Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-3/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

33. assistant (2026-06-18T10:22:48.150Z)

A subtlety: the original methods log `f"Failed to add video: {e}"` ? i.e. `<message>: <exception>`. My helper formats `f"{error_msg}: {e}"`, so `error_msg` must be the message WITHOUT the trailing `: {e}`. So for `add_video`, error_msg = `"Failed to add video"`. This preserves the exact output byte-for-byte. Now let me route each method.

34. user (2026-06-18T10:22:51.873Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-3/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

35. user (2026-06-18T10:22:57.808Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-3/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

36. user (2026-06-18T10:23:02.534Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-3/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

37. user (2026-06-18T10:23:08.616Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-3/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

38. user (2026-06-18T10:23:13.288Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-3/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

39. user (2026-06-18T10:23:17.670Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-3/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

40. user (2026-06-18T10:23:22.315Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-3/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

41. user (2026-06-18T10:23:27.368Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-3/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

42. user (2026-06-18T10:23:35.332Z)

```
The file C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/wf_1d39e81c-851-3/backend/infrastructure/database.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

43. user (2026-06-18T10:23:40.626Z)

```
The file C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/wf_1d39e81c-851-3/backend/infrastructure/database.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

44. user (2026-06-18T10:23:47.092Z)

```
The file C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/wf_1d39e81c-851-3/backend/infrastructure/database.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

45. assistant (2026-06-18T10:23:49.936Z)

```
All 11 routed. Let me verify the excluded methods are untouched and confirm there are no
leftover `return True` bool-writers I missed by re-reading the relevant region.
```

46. user (2026-06-18T10:23:51.329Z)

```
44:    def _execute_write(self, sql: str, params: Tuple[Any, ...], error_msg: str) -> bool:
47:        Captures the connect/commit/except-logger.error/return-False boilerplate shared by
54:                conn.cursor().execute(sql, params)
56:            return True
58:            logger.error(f"{error_msg}: {e}")
297:        return self._execute_write(
352:            logger.error(f"Failed to add play segment: {e}")
356:        return self._execute_write(
382:            logger.error(f"Failed to add candidate segment: {e}")
387:        return self._execute_write(
410:            rows = conn.cursor().execute(query, params).fetchall()
420:            row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
479:            conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
483:    # Writers follow the repo convention (swallow + logger.error, return bool); the
492:        return self._execute_write(
502:        return self._execute_write(
509:        return self._execute_write(
515:        return self._execute_write(
523:        return self._execute_write(
532:        return self._execute_write(
541:            row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
(job_id,)).fetchone()
553:        return self._execute_write(
563:        return self._execute_write(
577:                row = conn.cursor().execute(
585:            logger.error(f"Failed to compute next-due delay: {e}")
610:            logger.error(f"Failed to claim a due job: {e}")
626:            logger.error(f"Failed to sweep stale jobs: {e}")
695:            logger.error(f"Failed to upsert user_model (user={user_id}, v={version}): {e}")
704:                row = conn.cursor().execute(
707:                row = conn.cursor().execute(
```

47. assistant (2026-06-18T10:23:55.421Z)

```
All 11 routed methods now use `_execute_write`. The remaining `logger.error` lines are the
excluded methods (add_play_segment, add_candidate_segment lastrowid; fail_stale_jobs int;
seconds_until_next_due/claim_due_job reads; upsert_user_model lastrowid) ? all correctly
untouched. `prune_segments`, `clear_video_data`, `segment_watermarks` (no error handling /
```

multi-statement / reads) untouched.

Now re-verify the same characterization tests pass unchanged, plus the full database test file.

48. user (2026-06-18T10:23:58.458Z)

```
.........................                                                [100%]
27 passed in 0.99s
```

49. assistant (2026-06-18T10:24:01.702Z)

All 27 pass post-refactor (unchanged). Let me run the related async/job/execution-policy tests that exercise these writers in success paths, then the full gate.

50. user (2026-06-18T10:24:14.642Z)

```
.......................................................................... [ 97%]
..                                                                       [100%]
74 passed in 9.56s
```

51. assistant (2026-06-18T10:24:17.284Z)

All green. Now the full binding gate. Step 1: full suite.

52. user (2026-06-18T10:25:27.909Z)

```
 Full test suite:  C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest
-q
 obsreport (optional reporting dep): available
========================================================================
.....................ss.................................................. [  8%]
.......................................................................... [ 16%]
.......................................................................... [ 24%]
.......................................................................... [ 33%]
.......................................................................... [ 41%]
.......................................................................... [ 49%]
.......................................................................... [ 57%]
.......................................................................... [ 66%]
.......................................................................... [ 74%]
..............sssss...................................................... [ 82%]
.......................................................................... [ 91%]
.......................................................................... [ 99%]
......                                                                   [100%]
863 passed, 7 skipped in 66.89s (0:01:06)
========================================================================
 RESULT: PASS - all tests green
========================================================================
```

53. assistant (2026-06-18T10:25:30.666Z)

863 passed, 7 skipped. Now ruff, mypy, and coverage.

54. user (2026-06-18T10:26:12.743Z)

```
=== RUFF ===
All checks passed!
=== MYPY ===
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
```

```
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 107 source files
```

55. assistant (2026-06-18T10:26:15.273Z)

```
Ruff clean, mypy clean (those are pre-existing informational notes, "Success: no issues found").
Now coverage gate.
```

56. user (2026-06-18T10:27:58.571Z)

```
backend\reporting\run_signals.py                      14      0   100%
backend\results.py                                    27      0   100%
backend\sports\__init__.py                            17      1    94%
backend\sports\badminton.py                           13      1    92%
backend\sports\base.py                                12      3    75%
backend\tools\__init__.py                              0      0   100%
backend\tools\export.py                               75      2    97%
backend\tools\rally_slicer.py                        104     47    55%
backend\utils\__init__.py                              0      0   100%
backend\utils\geometry.py                             29      0   100%
backend\utils\hashing.py                               9      0   100%
backend\utils\logging_setup.py                        11      0   100%
backend\utils\paths.py                                27      3    89%
backend\utils\run_telemetry.py                       126     12    90%
backend\utils\video.py                                25      3    88%
training\__init__.py                                   0      0   100%
training\gen0\__init__.py                              0      0   100%
training\gen0\harness.py                             108     11    90%
----------------------------------------------------------------------
TOTAL                                               8024   1509    81%
Required test coverage of 70% reached. Total coverage: 81.19%
863 passed, 7 skipped, 7 warnings in 98.46s (0:01:38)
======================================================================
 RESULT: PASS - all tests green
======================================================================
```

57. assistant (2026-06-18T10:28:02.296Z)

```
All four gates green. Coverage is 81.19%, which is above the ~80.9% base ? not a regression (the
dedup plus 6 new failure-branch tests improved database.py coverage). Let me confirm the
database.py module coverage specifically and check the diff.
```

58. user (2026-06-18T10:28:04.587Z)

```
 backend/infrastructure/database.py | 200 +++++++++++++++----------------------
 tests/test_database.py             |  98 ++++++++++++++++++
 2 files changed, 177 insertions(+), 121 deletions(-)
===NUMSTAT===
79      121     backend/infrastructure/database.py
98      0       tests/test_database.py
```

59. assistant (2026-06-18T10:28:06.973Z)

```
Now let me verify HEAD is still my branch with only my changes vs origin/master before
committing (shared-checkout safety).
```

60. user (2026-06-18T10:28:09.716Z)

```
BRANCH:
refactor/db-execute-write-helper
HEAD:
e67add1 refactor(api): extract chunked-upload island from main into backend.api.uploads router
(#228)
COMMITS AHEAD OF origin/master:
```

```
STATUS:
 M backend/infrastructure/database.py
 M tests/test_database.py
```

61. assistant (2026-06-18T10:28:13.310Z)

```
HEAD == origin/master (no sibling commits), branch correct, only my two files modified. Stage
explicit paths and commit.
```

62. user (2026-06-18T10:28:14.650Z)

```
 backend/infrastructure/database.py | 200 +++++++++++++++----------------------
 tests/test_database.py             |  98 ++++++++++++++++++++
 2 files changed, 177 insertions(+), 121 deletions(-)
```

63. user (2026-06-18T10:28:24.397Z)

```
[refactor/db-execute-write-helper f7b8a78] refactor(db): route bool-returning single-statement
writers through _execute_write
 2 files changed, 177 insertions(+), 121 deletions(-)
---
f7b8a78 refactor(db): route bool-returning single-statement writers through _execute_write
```

64. assistant (2026-06-18T10:28:26.442Z)

```
Now push and open the PR.
```

65. user (2026-06-18T10:28:29.804Z)

```
remote:
remote: Create a pull request for 'refactor/db-execute-write-helper' on GitHub by visiting:
remote:      https://github.com/avidullu/badminton-highlight-indexer/pull/new/refactor/db-
execute-write-helper
remote:
branch 'refactor/db-execute-write-helper' set up to track 'origin/refactor/db-execute-write-
helper'.
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]      refactor/db-execute-write-helper -> refactor/db-execute-write-helper
```

66. user (2026-06-18T10:28:46.121Z)

```
https://github.com/avidullu/badminton-highlight-indexer/pull/232
```

67. assistant (2026-06-18T10:28:49.589Z)

```
PR opened: https://github.com/avidullu/badminton-highlight-indexer/pull/232

Let me capture the final diffstat for the structured report.
```

68. user (2026-06-18T10:28:51.243Z)

```
 backend/infrastructure/database.py | 200 +++++++++++++++----------------------
 tests/test_database.py             |  98 ++++++++++++++++++++
 2 files changed, 177 insertions(+), 121 deletions(-)
```

69. assistant (2026-06-18T10:28:53.121Z)

```
All four gates green and the PR is open. Returning the structured result.
```

70. user (2026-06-18T10:29:10.056Z)

```
Structured output provided successfully
```

71. assistant (2026-06-18T10:29:13.739Z)

Done. The refactor is complete and PR #232 is open.

Summary: Introduced a private `_execute_write(self, sql, params, error_msg) -> bool` helper in `backend/infrastructure/database.py` and routed the 11 bool-returning single-statement writers through it, preserving each method's exact log message verbatim. Characterization tests were written first (6 write-failure tests pinning the exact per-method log text), confirmed passing on unchanged code, then re-verified passing after the dedup. All four gates green (863 passed/7 skipped, ruff clean, mypy clean, coverage 81.19% ? no regression vs ~80.9% base).